



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Solid

Solid Principles (Bag. 1)

Pemrograman Berorientasi Objek

SOLID

- Single Responsibility Principle (SRP)
- Open/Closed Principle (OCP)
- Liskov's Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)



Bad Designs

Rigidity

It is hard to change because every change affects too many other parts of the system

Fragility

When you make a change, unexpected parts of the system break

Immobility

It is hard to reuse in another application because it cannot be disentangled from the current application

Single Responsibility Principle

a **class** should be having one and only one responsibility

a **class** should have one and only one reason to change

Single Responsibility Principle

```
1 public class Employee {  
2     public calculatePay(Money m) {  
3         // business logic for payment  
4     }  
5  
6     public Employee save(Employee e) {  
7         // store employee  
8     }  
9 }
```



Single Responsibility Principle

```
1 public class Employee {  
2     public calculatePay(Money m) {  
3         // business logic for payment  
4     }  
5 }
```

```
1 public class EmployeeRepo {  
2     public Employee save(Employee e) {  
3         // store employee  
4     }  
5 }
```



Example (Cont.)

```
public class AreaCalculator {
    public int sum(List<Object> shapes) {
        int sum = 0;
        for (int i = 0; i < shapes.size(); i++) {
            Object shape = shapes.get(i);
            if (shape instanceof Circle) {
                sum += Math.PI * Math.pow(((Circle) shape).getRadious(), b:2);
            } else if (shape instanceof Square) {
                sum += Math.pow(((Square) shape).getLength(), b:2);
            }
        }
        return sum;
    }

    public String json(List<Object> shapes) {
        return "{sum: %s}".formatted(sum(shapes));
    }

    public String csv(List<Object> shapes) {
        return "sum,%s".formatted(sum(shapes));
    }
}
```



```

src > com > solidtutorial > Main.java > Main > main(String[])
1  package com.solidtutorial;
2
3  import java.util.List;
4
5  public class Main {
6      Run | Debug
7      public static void main(String[] args) {
8          AreaCalculator areaCalculator = new AreaCalculator();
9          Circle circle = new Circle(radius:5);
10         Square rectangle = new Square(length:5);
11
12         List<Object> shapes = List.of(circle, rectangle);
13         int sum = areaCalculator.sum(shapes);
14         System.out.println(areaCalculator.json(shapes));
15         System.out.println(areaCalculator.csv(shapes));
16     }
17

```

```

>
> cd /Users/galuhdipabharata/Developer/00P/SOLID ; /usr/bin/env /Users/galuhdipabharata/.sdkman/candidates/java/17.0.9-amzn/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/galuhdipabharata/Library/Application\ Support/Code/User/workspaceStorage/1569b9166e28afe13ee1e403fad766e3/redhat.java/jdt_ws/SOLID_d6c6c0ea/bin com.solidtutorial.Main

{sum: 103}
sum,103

~/Developer/00P/SOLID ..... 08:51:37
>

```




```
package com.solidtutorial;
import java.util.List;

✓ public class AreaCalculator {
✓     public int sum(List<Object> shapes) {
        int sum = 0;
        for (int i = 0; i < shapes.size(); i++) {
            Object shape = shapes.get(i);
            if (shape instanceof Circle) {
                sum += Math.PI * Math.pow(((Circle) shape).getRadiious(),
                    b:2);
            } else if (shape instanceof Square) {
                sum += Math.pow(((Square) shape).getLength(), b:2);
            }
        }
        return sum;
    }
}
```

```
src > com > solidtutorial > ShapesPrinter.java > ...
1  package com.solidtutorial;
2
3  public class ShapesPrinter {
4
5      public String json(int sum) {
6          return "{shapes_sum: %s}".formatted(sum);
7      }
8
9      public String csv(int sum) {
10         return "shapes_sum,%s".formatted(sum);
11     }
12 }
13
```

> com > solidtutorial > Main.java > ...

```

1 package com.solidtutorial;
2
3 import java.util.List;
4
5 public class Main {
6     Run | Debug
7     public static void main(String[] args) {
8         AreaCalculator areaCalculator = new AreaCalculator();
9         Circle circle = new Circle(radius:5);
10        Square rectangle = new Square(length:5);
11        ShapesPrinter shapesPrinter = new ShapesPrinter();
12
13        List<Object> shapes = List.of(circle, rectangle);
14        int sum = areaCalculator.sum(shapes);
15        System.out.println(shapesPrinter.json(sum));
16        System.out.println(shapesPrinter.csv(sum));
17    }
18 }

```

```

>
> cd /Users/galuhdipabharata/Developer/00P/
SOLID ; /usr/bin/env /Users/galuhdipabharata
.sdkman/candidates/java/17.0.9-amzn/bin/jav
a -XX:+ShowCodeDetailsInExceptionMessages -c
p /Users/galuhdipabharata/Library/Applicatio
n\ Support/Code/User/workspaceStorage/1569b9
166e28afe13ee1e403fad766e3/redhat.java/jdt_w
s/SOLID_d6c6c0ea/bin com.solidtutorial.Main

```

```

{sum: 103}
sum,103

```

```

>
>
> cd /Users/galuhdipabharata/Developer/00P/
SOLID ; /usr/bin/env /Users/galuhdipabharata
.sdkman/candidates/java/17.0.9-amzn/bin/jav
a -XX:+ShowCodeDetailsInExceptionMessages -c
p /Users/galuhdipabharata/Library/Applicatio
n\ Support/Code/User/workspaceStorage/1569b9
166e28afe13ee1e403fad766e3/redhat.java/jdt_w
s/SOLID_d6c6c0ea/bin com.solidtutorial.Main

```

```

{shapes_sum: 103}
shapes_sum,103

```



Open/Closed Principle

Software entities (classes, modules, functions) should be open for extension, but closed for modification

- **Open:** add data members and operations through inheritance
- **Close:** available for use by other components but may not, itself, be changed



Open/Closed Principle

```
1 public class PaymentManager {  
2     private PaymentType ptype;  
3  
4     public void pay(Money m) {  
5         if (ptype==PaymentType.Cash) {  
6             // pay with cash  
7         }  
8         else {  
9             // pay with credit card  
10        }  
11    }  
12 }
```



Open/Closed Principle

```
1 public abstract class Payment {  
2     public abstract void pay(Money m);  
3 }  
4  
5 public class CashPayment extends Payment {  
6     public void pay(Money m) {  
7         // pay with cash  
8     }  
9 }  
10  
11 public class CreditCardPayment extends Payment {  
12     public void pay(Money m) {  
13         // pay with credit card  
14     }  
15 }
```



```
src / com / solidtutorial / Cube.java / ...  
1  package com.solidtutorial;  
2  
3  public class Cube {  
4      private final int length;  
5  
6      public Cube(int length) {  
7          this.length = length;  
8      }  
9  
10     public int getLength() {  
11         return length;  
12     }  
13 }  
14
```



```
public class AreaCalculator {  
    public int sum(List<Object> shapes) {  
        int sum = 0;  
        for (int i = 0; i < shapes.size(); i++) {  
            Object shape = shapes.get(i);  
            if (shape instanceof Circle) {  
                sum += Math.PI * Math.pow(((Circle) shape).getRadius(),  
                    b:2);  
            }  
            if (shape instanceof Square) {  
                sum += Math.pow(((Square) shape).getLength(), b:2);  
            }  
            if (shape instanceof Cube) {  
                sum += Math.pow(((Cube) shape).getLength(), b:3);  
            }  
        }  
        return sum;  
    }  
}
```



```
src > com > solidtutorial > Shape.java > ...
```

```
1 package com.solidtutorial;  
2  
3 public interface Shape {  
4     double area();  
5 }  
6
```

```
public class Circle implements Shape {  
    private final int radius;  
  
    public Circle(int radius) {  
        this.radius = radius;  
    }  
  
    public int getRadius() {  
        return radius;  
    }  
  
    @Override  
    public double area() {  
        return Math.PI * radius * radius;  
    }  
}
```

```
package com.solidtutorial;
```

```
public class Square implements Shape {  
    private final int length;  
  
    public Square(int length) {  
        this.length = length;  
    }  
  
    public int getLength() {  
        return length;  
    }  
  
    @Override  
    public double area() {  
        return length * length;  
    }  
}
```




```
package com.solidtutorial;

public class Cube implements Shape{
    private final int length;

    public Cube(int length) {
        this.length = length;
    }

    public int getLength() {
        return length;
    }

    @Override
    public double area() {
        return 6 * length * length;
    }
}
```



```
public class AreaCalculator {
    public int sum(List<Object> shapes) {
        int sum = 0;
        for (int i = 0; i < shapes.size(); i++) {
            Object shape = shapes.get(i);
            if (shape instanceof Circle) {
                sum += Math.PI * Math.pow(((Circle) shape).getRadius(),
                    b:2);
            }
            if (shape instanceof Square) {
                sum += Math.pow(((Square) shape).getLength(), b:2);
            }
            if (shape instanceof Cube) {
                sum += Math.pow(((Cube) shape).getLength(), b:3);
            }
        }
        return sum;
    }
}
```

```
package com.solidtutorial;
import java.util.List;

public class AreaCalculator {
    public int sum(List<Shape> shapes) {
        int sum = 0;
        for (int i = 0; i < shapes.size(); i++) {
            sum += shapes.get(i).area();
        }
        return sum;
    }
}
```

```

package com.solidtutorial;

import java.util.List;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        AreaCalculator areaCalculator = new AreaCalculator();
        Circle circle = new Circle(radius:5);
        Square rectangle = new Square(length:5);
        ShapesPrinter shapesPrinter = new ShapesPrinter();
        Cube cube = new Cube(length:5);

        List<Shape> shapes = List.of(circle, rectangle, cube);

        int sum = areaCalculator.sum(shapes);
        System.out.println(shapesPrinter.json(sum));
        System.out.println(shapesPrinter.csv(sum));
    }
}

```

```

> cd /Users/galuhdipabharata/Developer/00P/
SOLID ; /usr/bin/env /Users/galuhdipabharata
/.sdkman/candidates/java/17.0.9-amzn/bin/jav
a -XX:+ShowCodeDetailsInExceptionMessages -c
p /Users/galuhdipabharata/Library/Applicatio
n\ Support/Code/User/workspaceStorage/1569b9
166e28afe13ee1e403fad766e3/redhat.java/jdt_w
s/SOLID_d6c6c0ea/bin com.solidtutorial.Main

```

```

{shapes_sum: 253}
shapes_sum,253

```

```

~/Developer/00P/SOLID ..... 09:43:50
> 

```

Terima Kasih



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Solid

Solid Principles (Bag. 2)

Pemrograman Berorientasi Objek



Liskov's Substitution Principle

a subclass should behave in such a way that it will not cause problems when uses instead of the superclass

- references to some object (base class) must be able to use objects of classes derived from the base without any knowledge specialized to the derived classes



Liskov's Substitution Principle

```
1 public class CasualEmployee extends Employee {  
2     public String getProjectDetails(int empId) {  
3         super.getProjectDetails(empId);  
4         System.out.println("Casual Employee Project Detail");  
5     }  
6 }  
7  
8 public class ContractEmployee extends Employee {  
9     public String getProjectDetails(int empId) {  
10        // broken the base class  
11        System.out.println("Contract Employee Project Detail");  
12    }  
13 }
```



Substitution Failures

Deriving a square from a rectangle implies that one of the state variables, height or width, is redundant

The behavior of a square - change its height and you change its width - does not apply to rectangles and so square objects are not rectangle objects


```
package com.solidtutorial;
```

```
public class Rectangle {  
    protected int width;  
    protected int height;  
  
    public Rectangle(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
  
    public double area() {  
        return width * height;  
    }  
  
    public int getWidth() {  
        return width;  
    }  
  
    public int getHeight() {  
        return height;  
    }  
  
    public void setWidth(int width) {  
        this.width = width;  
    }  
  
    public void setHeight(int height) {  
        this.height = height;  
    }  
}
```

```
package com.solidtutorial;
```

```
public class OtherSquare extends Rectangle {  
    public OtherSquare(int side) {  
        super(side, side); // Calls Rectangle constructor with same value  
                             for width and height  
    }  
  
    @Override  
    public void setWidth(int width) {  
        this.width = width;  
        this.height = width;  
    }  
  
    @Override  
    public void setHeight(int height) {  
        this.width = height;  
        this.height = height;  
    }  
}
```



```

package com.solidtutorial;

import java.util.List;

public class Main {
    public static void processRectangle(Rectangle r) {
        r.setWidth(width:5);
        r.setHeight(height:4);
        System.out.println(r.area());
    }

    Run | Debug
    public static void main(String[] args) {
        AreaCalculator areaCalculator = new AreaCalculator();
        Circle circle = new Circle(radius:5);
        Square rectangle = new Square(length:5);
        ShapesPrinter shapesPrinter = new ShapesPrinter();
        Cube cube = new Cube(length:5);

        List<Shape> shapes = List.of(circle, rectangle, cube);

        int sum = areaCalculator.sum(shapes);
        System.out.println(shapesPrinter.json(sum));
        System.out.println(shapesPrinter.csv(sum));

        Rectangle rectangle2 = new Rectangle(width:5, height:4);
        OtherSquare otherSquare = new OtherSquare(side:5);
        processRectangle(rectangle2);
        processRectangle(otherSquare);
    }
}

```

```

> cd /Users/galuhdipabharata/Developer/00P/SOLID ; /usr/bin/env /Users/galuhdipabharata/.sdkman/candidates/java/17.0.9-amzn/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/galuhdipabharata/Library/Application\ Support/Code/User/workspaceStorage/1569b9166e28afe13ee1e403fad766e3/redhat.java/jdt_ws/SOLID_d6c6c0ea/bin com.solidtutorial.Main {shapes_sum: 253}
shapes_sum,253
20.0
16.0

```

```
~/Developer/00P/SOLID ..... 17:20:34
```

```
> []
```

```
package com.solidtutorial;

public class Rectangle implements Shape{
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double area() {
        return width * height;
    }

    public int getWidth() {
        return width;
    }

    public int getHeight() {
        return height;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }
}
```

```
package com.solidtutorial;

public class OtherSquare implements Shape {
    private int side;

    public OtherSquare(int side) {
        this.side = side;
    }

    public void setSide(int side) {
        this.side = side;
    }

    @Override
    public double area() {
        return side * side;
    }
}
```



```
package com.solidtutorial;
```

```
import java.util.List;
```

```
public class Main {
    public static void processRectangle(Shape r) {
        // r.setWidth(5);
        // r.setHeight(4);
        System.out.println(r.area());
    }
}
```

Run | Debug

```
public static void main(String[] args) {
    AreaCalculator areaCalculator = new AreaCalculator();
    Circle circle = new Circle(radius:5);
    Square rectangle = new Square(length:5);
    ShapesPrinter shapesPrinter = new ShapesPrinter();
    Cube cube = new Cube(length:5);

    List<Shape> shapes = List.of(circle, rectangle, cube);

    int sum = areaCalculator.sum(shapes);
    System.out.println(shapesPrinter.json(sum));
    System.out.println(shapesPrinter.csv(sum));

    // Rectangle rectangle2 = new Rectangle(5, 4);
    // OtherSquare otherSquare = new OtherSquare(5);
    // processRectangle(rectangle2);
    // processRectangle(otherSquare);

    Rectangle rect = new Rectangle(width:5, height:4);
    OtherSquare otherSquare = new OtherSquare(side:5);
    processRectangle(rect);
    processRectangle(otherSquare);
}
```

```
> cd /Users/galuhdipabharata/Developer/OOP/
SOLID ; /usr/bin/env /Users/galuhdipabharata
.sdkman/candidates/java/17.0.9-amzn/bin/jav
a -XX:+ShowCodeDetailsInExceptionMessages -c
p /Users/galuhdipabharata/Library/Applicatio
n\ Support/Code/User/workspaceStorage/1569b9
166e28afe13ee1e403fad766e3/redhat.java/jdt_w
s/SOLID_d6c6c0ea/bin com.solidtutorial.Main
```

```
{shapes_sum: 253}
shapes_sum,253
20.0
25.0
```

```
~/Developer/00P/SOLID ..... 19:12:41
> █
```

Interface Segregation Principle

Clients should not be forced to depend upon interfaces they do not use

- When a component has several different clients it is tempting to provide a large interface that satisfies the needs of all clients
- It is much better design to have the component support multiple interfaces, one appropriate for each client



Interface Segregation Principle

```
1 public interface IEmployee {  
2     public String getProjectDetails*int empId);  
3     public String getEmployeeDetails*int empId);  
4 }  
5  
6 public class ContractEmployee extends IEmployee {  
7     public String getProjectDetails(int empId) {  
8         // code for specific project detail  
9         System.out.println("Contract Employee Project Detail");  
10    }  
11    public String getEmployeeDetails*int empId) {  
12        // we don't need this methode  
13        throw new NotImplementedException();  
14    }  
15 }
```



```
package com.solidtutorial;

public class Cube implements Shape {
    private final int length;

    public Cube(int length) {
        this.length = length;
    }

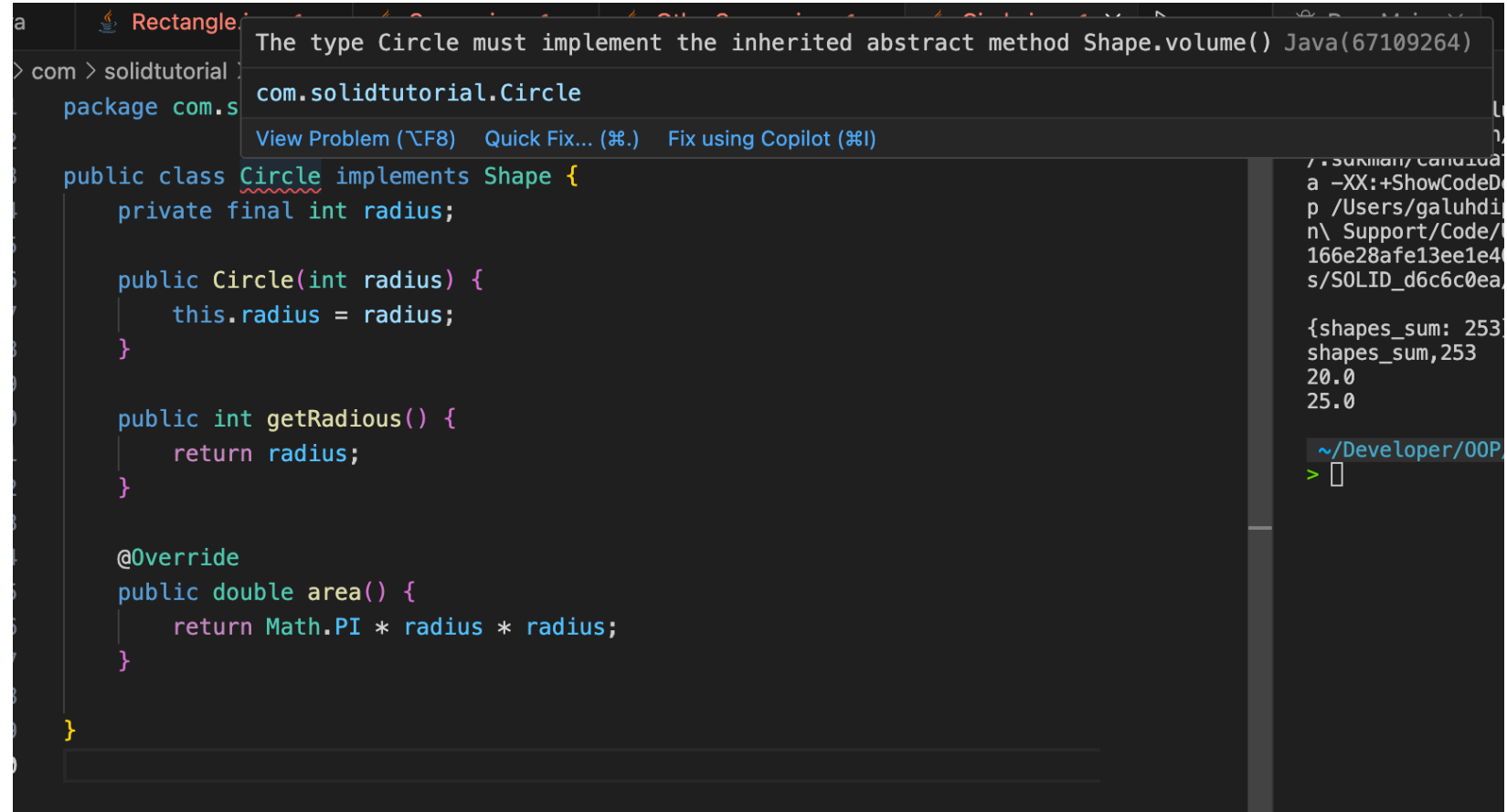
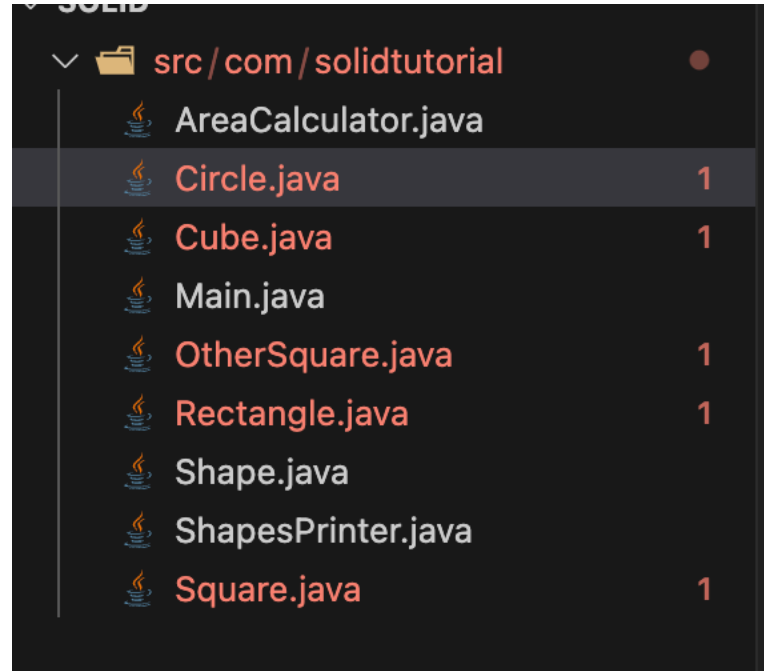
    public int getLength() {
        return length;
    }

    @Override
    public double area() {
        return 6 * length * length;
    }
}
```

```
package com.solidtutorial;

public interface Shape {
    double area();
    double volume();
}
```






```
src > com > solidtutorial > ☕ ThreeDimensionalShape.java > ...
```

```
1 package com.solidtutorial;  
2  
3 public interface ThreeDimensionalShape {  
4     double volume();  
5 }  
6
```

```
package com.solidtutorial;  
  
public class Cube implements Shape, ThreeDimensionalShape {  
    private final int length;  
  
    public Cube(int length) {  
        this.length = length;  
    }  
  
    public int getLength() {  
        return length;  
    }  
  
    @Override  
    public double area() {  
        return 6 * length * length;  
    }  
  
    @Override  
    public double volume() {  
        return length * length * length;  
    }  
}
```



Dependency Inversion Principle

High level components should not depend upon low level components. Instead, both should depend on abstractions

Abstractions should not depend upon details. Details should depend upon the abstractions

Dependency Inversion Principle

```
1 ▼ public class Notification {  
2     private Email email;  
3 ▼   public Notification() {  
4       email = new Email();  
5     }  
6 ▼   public void promoNotification() {  
7       email.send();  
8     }  
9 }
```



Dependency Inversion Principle

```
1 public class Notification {  
2     private Messenger mess;  
3     public Notification(Messenger mess) {  
4         this.mess = mess;  
5     }  
6     public void promoNotification() {  
7         mess.send();  
8     }  
9 }
```



```
package com.solidtutorial;

import java.util.List;

public class ShapesPrinter {

    private AreaCalculator areaCalculator = new AreaCalculator();

    public String json(List<Shape> shapes) {
        return "{shapes_sum: %s}".formatted(areaCalculator.sum(shapes));
    }

    public String csv(List<Shape> shapes) {
        return "shapes_sum,%s".formatted(areaCalculator.sum(shapes));
    }
}
```



src > com > solidtutorial >  IAreaCalculator.java > ...

```
1  package com.solidtutorial;
2
3  import java.util.List;
4
5  public interface IAreaCalculator {
6      int sum(List<Shape> shapes);
7  }
8
```

src > com > solidtutorial >  AreaCalculator.java >  AreaCalculator

```
1  package com.solidtutorial;
2  import java.util.List;
3
4  public class AreaCalculator implements IAreaCalculator {
5      public int sum(List<Shape> shapes) {
6          int sum = 0;
7          for (int i = 0; i < shapes.size(); i++) {
8              sum += shapes.get(i).area();
9          }
10         return sum;
11     }
12 }
```

```
package com.solidtutorial;

import java.util.List;

public class ShapesPrinter {

    private final IAreaCalculator areaCalculator;

    public ShapesPrinter(IAreaCalculator areaCalculator) {
        this.areaCalculator = areaCalculator;
    }

    public String json(List<Shape> shapes) {
        return "{shapes_sum: %s}".formatted(areaCalculator.sum(shapes));
    }

    public String csv(List<Shape> shapes) {
        return "shapes_sum,%s".formatted(areaCalculator.sum(shapes));
    }
}
```



Terima Kasih